Skeleton Loader: Machine Coding Walkthrough

- **Problem:** Build a Skeleton Loader (placeholder UI shown while data loads)
- **Tech stack:** React (Vite, JavaScript)
- **Time limit:** 60 minutes

PART A: Thought process

These are the general steps for any machine coding task:

1. **Clarify.** Restate the problem, ask questions, and write down assumptions.
2. **List interactions.** Every user action and its expected result. This becomes the checklist.
3. **Sketch components.** Who renders what, and who owns which state.
4. **Model state and data.** Store the minimum and derive the rest.
5. **Find the hard parts early.** Solve them on paper before you type.
6. **Build in layers.** Get a working version first, then add one layer at a time, and keep it runnable after each step.
7. **Cover states and edge cases.** Loading, success, empty, error, plus problem-specific ones.
8. **Wrap up.** Trade-offs, what you would add with more time.

Time budget (60 min)

Phase	Time
Requirements + clarifying questions	5 min
Interactions, components, state on paper	8 min
Generic `Skeleton` primitive + CSS animation	10 min
Mock API + fetch hook (loading / success / error / empty)	10 min
Card skeleton that mirrors the real card (no layout shift)	10 min
Delay + minimum duration (no flash, no flicker)	7 min
Accessibility, reduced motion, polish	5 min
Wrap-up and trade-offs	5 min

> **Think out loud.** The interviewer grades the reasoning as much as the code. Say *why* before *what*: "I'll animate `transform` instead of `background-position` because it doesn't repaint each frame."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A reusable `Skeleton` component with variants: **text** (one or many lines), **circle** (avatar), and **rect** (image / block).
- Configurable `width`, `height`, `lines`, and `animation` (`shimmer` | `pulse` | `none`).
- A **composite skeleton** (`UserCardSkeleton`) that has the same shape as the real `UserCard`.
- A demo page that fetches a list of users and shows the skeleton while loading, then the real cards.
- Handle **error** (with Retry) and **empty** results.
- Switch between scenarios (fast / slow / empty / error) to demo every state.

Non-functional requirements

- **Performance:** the animation must be cheap (GPU / compositor only), and swapping skeleton → content must cause **no layout shift (CLS)**.
- **Usability:** no skeleton "flash" for fast responses, and no flicker when a response arrives just after the skeleton appears.
- **Accessibility:** screen readers hear "Loading users…" once, not dozens of empty boxes; respect `prefers-reduced-motion`.
- **Reusability:** the primitive knows nothing about users; composites are built from primitives.
- **Theming:** colors come from CSS variables (light and dark mode).
- **Error handling:** failed requests show a message and a Retry; stale requests are cancelled.

Clarifying questions

1. Is this a generic component library piece, or a skeleton for one specific screen? *(Assume both: a primitive plus one composite.)*
2. Which animation: shimmer, pulse, or both? Should it respect reduced motion?
3. Should fast responses (under ~200 ms) skip the skeleton entirely?
4. How many skeleton items should show if we don't know the result count yet?
5. Is there a real API, or can I mock one with configurable latency and failures?
6. Dark mode / theming needed?

Assumptions

- Mock API with latency per scenario; 6 users per page.
- Skeleton appears only if loading lasts > **200 ms**, and once shown stays at least **500 ms**.
- Show **6** skeleton cards (the page size) so the grid doesn't jump.
- No external libraries: plain React + CSS.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads (slow API)	Blank reserved area for 200 ms, then 6 skeleton cards with shimmer, then real cards
Page loads (fast API, 100 ms)	Skeleton never appears; cards render directly (no flash)
Response arrives 250 ms after start	Skeleton appears at 200 ms and stays until 700 ms (min 500 ms), then cards (no flicker)
API returns `[]`	"No users found." message
API fails	Error message with **Retry** button (`role="alert"`)
Click Retry / Reload	Back to loading → skeleton → result
Switch scenario while loading	Old request aborted; only the new result is shown
Change animation to Pulse / None	Skeletons pulse / stay static
OS has reduced motion on	No animation, static grey blocks
Screen reader during load	Hears "Loading users…" once; skeleton blocks are hidden from it
Skeleton → content swap	Card boxes keep the same size: no layout shift

3. Design the architecture

Module	Responsibility
`usersApi.js`	Mock `fetchUsers({ scenario, signal })` with latency, empty, error, abort
`useUsers`	Owns fetch state (`status` , `data` , `error`), aborts stale requests, exposes `reload`
`useDeferredLoading`	Turns raw `loading` into `showSkeleton` with a delay + minimum duration
`Skeleton`	Generic placeholder primitive (text / circle / rect, animation)
`UserCardSkeleton`	Composite: arranges `Skeleton` s in the exact layout of `UserCard`
`UserCard`	Real content card
`App`	Controls (scenario, animation, reload) and picks which state to render

```
App
├── controls (scenario select, animation select, Reload)
└── section.results [aria-busy]
    ├── span[role=status]  "Loading users…"
    ├── showSkeleton → ul.grid > UserCardSkeleton × 6
    │                    └── Skeleton (circle, text, text, text×2)
```

```
    ├── error        → div[role=alert] + Retry
    ├── empty        → "No users found."
    └── success      → ul.grid > UserCard × n

useUsers(scenario) ──> fetchUsers()      (data)
useDeferredLoading(status === 'loading') (timing)
```

Start simple: one `Skeleton` and the `App`. Split out `useDeferredLoading` only once the timing logic appears.

---

4. Design the state / data model

State or field	Owner	Purpose
`scenario`	`App`	Which mock latency / outcome to use
`animation`	`App`	`shimmer` / `pulse` / `none` passed to skeletons
`status`	`useUsers`	`loading` / `success` / `error` (single source of truth)
`data`	`useUsers`	The user list
`error`	`useUsers`	Error message to show
`reloadKey`	`useUsers`	Bumping it re-runs the fetch effect (Retry / Reload)
`show`	`useDeferredLoading`	Whether the skeleton is visible right now
`shownAt` (ref)	`useDeferredLoading`	When the skeleton appeared; a ref because it never affects render

```
const LOADING = { status: 'loading', data: [], error: null };
const [state, setState] = useState(LOADING);   // useUsers
const [show, setShow] = useState(false);        // useDeferredLoading
const shownAt = useRef(0);
```

Senior note:

- `isEmpty` is **derived** (`status === 'success' && data.length === 0`), not stored.
- `busy` is **derived** (`status === 'loading' || showSkeleton`) and drives `aria-busy` and the disabled Reload.
- One `status` enum instead of `isLoading` + `isError` booleans: impossible combinations like loading-and-error can't exist.
- Fetch state lives in the hook, timing state in another hook; `App` only composes them.

---

5. Design the data structures and contracts

Example data

```
{ id: 'u1', name: 'Aarav Sharma', role: 'Frontend Engineer', bio: 'Builds design systems…' }
```

Stable `id`s are used as React keys for real cards. Skeleton items use the index as key, which is fine: they are static, identical, and never reordered.

API contract (mock)

```
fetchUsers({ scenario: 'fast' | 'slow' | 'empty' | 'error', signal?: AbortSignal })
  → Promise<User[]>
  // resolves after SCENARIOS[scenario].delay ms
  // 'empty' → []
  // 'error' → rejects Error('Could not load users. Please try again.')
  // signal aborted → rejects DOMException name 'AbortError'
```

Component contracts

```
<Skeleton
  variant="text | circle | rect"   // default 'text'
  width={number | string}          // 48, '55%'
  height={number | string}         // ignored for text: text uses line-height
  lines={number}                   // text only, default 1
  animation="shimmer | pulse | none"
  className="…"                    // lets the caller reuse the real element's typography
/>

useDeferredLoading(loading, { delay = 200, minDuration = 500 }) → boolean
useUsers(scenario) → { status, data, error, reload }
```

---

6. Trace the data flow

Interaction: user picks "Slow (2 s)"

```
select onChange → setScenario('slow')
        |
        ▼
useUsers effect cleanup → controller.abort()   (previous request dies)
```

```
useUsers effect → setState(LOADING) → fetchUsers({ scenario:'slow', signal })
        │
        ▼
status === 'loading' → useDeferredLoading starts a 200 ms timer
        │ (0–200 ms: nothing rendered, area keeps its min-height)
        ▼
200 ms: shownAt = now, show = true → App renders 6 UserCardSkeleton
        │ (shimmer runs on ::after with transform, aria-busy=true, "Loading users…")
        ▼
2000 ms: promise resolves → setState({ status:'success', data })
        │
        ▼
useDeferredLoading: shown for 1800 ms ≥ 500 → setShow(false) now
        │
        ▼
App renders 6 UserCard; same .card box, same line-heights → no layout shift
aria-busy=false, status text cleared
```

---

7. Important technical challenges

Challenge 1: No layout shift when the skeleton is replaced

Problem: a text skeleton like `height: 12px; margin: 6px 0` doesn't match a heading with `font-size: 16px; line-height: 1.4` (22.4 px). When the real card arrives, every card grows by a few pixels and the whole grid jumps. That jump is a CLS hit and looks cheap.

Solution:

- Composite skeleton reuses the **same CSS classes** as the real card (`.card` , `.card__header` , `.card__name` …), so padding, gaps and font sizes are identical.
- A text bar takes **exactly one line-height** of the inherited font: it contains a non-breaking space, and is then shrunk *visually* with `transform: scale(1, 0.6)` . Transform doesn't affect layout, so the box stays 22.4 px.
- Fixed-size media (avatar 48×48) and the same number of items as the page size.

```
.skeleton--text {
  height: auto;
  transform: scale(1, 0.6);   /* visual only, layout keeps 1 line-height */
  transform-origin: 0 55%;
}
.skeleton--text::before { content: '\00a0'; }  /* one line of text */
```

```
<Skeleton className="card__name" width="55%" />  {/* inherits h2 typography */}
```

Challenge 2: No flash for fast responses, no flicker for medium ones

Problem: a 100 ms response flashes the skeleton for 100 ms: worse than showing nothing. A 250 ms response shows the skeleton for 50 ms and then swaps: a flicker.

Solution: separate "is loading" from "show skeleton". Wait `delay` (200 ms) before showing; once shown, keep it for at least `minDuration` (500 ms).

```
useEffect(() => {
  if (loading) {
    if (show) return;
    const t = setTimeout(() => { shownAt.current = Date.now(); setShow(true); }, delay);
    return () => clearTimeout(t);       // finished before delay → never shown
  }
  if (!show) return;
  const remaining = minDuration - (Date.now() - shownAt.current);
  const t = setTimeout(() => setShow(false), Math.max(0, remaining));
  return () => clearTimeout(t);
}, [loading, show, delay, minDuration]);
```

Challenge 3: A cheap, accessible animation

Problem: animating `background-position` on a gradient repaints every skeleton every frame. With 30 skeletons on a low-end phone that janks. Users with vestibular disorders may also get motion sickness from it.

Solution: put the gradient on a `::after` and animate **`transform: translateX`**: this runs on the compositor with no layout and no paint. Disable it under `prefers-reduced-motion`.

```
.skeleton { position: relative; overflow: hidden; background: var(--sk-base); }
.skeleton--shimmer::after {
  content: ''; position: absolute; inset: 0; transform: translateX(-100%);
  background: linear-gradient(90deg, transparent, var(--sk-highlight), transparent);
  animation: sk-shimmer 1.4s ease-in-out infinite;
}
@keyframes sk-shimmer { to { transform: translateX(100%); } }
@media (prefers-reduced-motion: reduce) {
  .skeleton--shimmer::after, .skeleton--pulse { animation: none; }
}
```

Challenge 4: Screen readers and stale requests

Problem: 6 cards × 5 blocks = 30 meaningless nodes read aloud. Separately, switching scenarios quickly can let an old slow response overwrite the newer one.

Solution: skeletons are `aria-hidden="true"`; the container gets `aria-busy` and a single visually hidden `role="status"` saying "Loading users…". Each request gets an `AbortController`; the effect cleanup aborts it, and `AbortError` is ignored.

```
<section className="results" aria-busy={busy}>
  <span className="sr-only" role="status">{busy ? 'Loading users…' : ''}</span>
  {content}
</section>
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite app, `App` with a heading	Page renders
2	`Skeleton` primitive (rect, circle, text) + base CSS	Static grey blocks
3	Shimmer + pulse animation, reduced motion	Animated blocks
4	Mock `fetchUsers` with scenarios	Promise that resolves / rejects
5	`useUsers` hook with `status` enum	loading / success / error in state
6	`UserCard` + grid; render success	Real data on screen
7	`UserCardSkeleton` with shared classes; render while loading	Skeleton → content
8	Fix text height via line-height trick, check no shift	Zero layout shift
9	Error (Retry) and empty states	All states visible
10	`AbortController` + Reload	No stale overwrites
11	`useDeferredLoading` (delay + min duration)	No flash, no flicker
12	`aria-busy`, `role=status`, `aria-hidden`, dark mode tokens	Accessible and themed

---

9. Cover all states and edge cases

State / edge case	What the UI does
Initial (loading < 200 ms)	Nothing inside the results area; `min-height` keeps the page from collapsing
Loading (> 200 ms)	6 skeleton cards, `aria-busy=true`, "Loading users…" for screen readers
Success	Real cards, same size as skeletons
Empty	"No users found."
Error	Message + Retry, `role="alert"`
Very fast response	Skeleton never shown
Response just after skeleton shows	Skeleton kept for 500 ms minimum
Scenario switched mid-request	Previous request aborted; its result ignored
Reload while loading	Button disabled (`busy`)
Real text longer than skeleton (3-line bio)	That card grows by one line: acceptable, skeleton matches the common case
Reduced motion	Static blocks
Dark mode	Skeleton colors from CSS variables
React StrictMode double effect	First request aborted by cleanup; only one result lands
Unmount during request	Cleanup aborts; no state update after unmount

10. Final structure

```
_39_skelton_loader/
├── index.html
├── package.json
└── src/
    ├── main.jsx
    ├── App.jsx                  # controls + state switch
    ├── App.css                  # tokens, layout, shared .card styles
    ├── api/
    │   └── usersApi.js          # mock fetch with scenarios + abort
    ├── hooks/
    │   ├── useUsers.js          # fetch state, abort, reload
    │   └── useDeferredLoading.js   # delay + minimum duration
    └── components/
        ├── Skeleton.jsx         # generic primitive
```

```
        ├── Skeleton.css            # variants + shimmer / pulse
        ├── UserCard.jsx            # real content
        └── UserCardSkeleton.jsx    # composite mirroring UserCard
```

11. Full working code

`src/api/usersApi.js`

```
// Mock API: each scenario has its own latency so every UI state can be demoed.
const USERS = [
  { id: 'u1', name: 'Aarav Sharma', role: 'Frontend Engineer', bio: 'Builds design systems and cares about every pixel of the loading experience.' },
  { id: 'u2', name: 'Diya Patel', role: 'Product Designer', bio: 'Turns rough ideas into flows people actually enjoy using.' },
  { id: 'u3', name: 'Kabir Singh', role: 'Backend Engineer', bio: 'Keeps the APIs fast so the skeletons do not stay long.' },
  { id: 'u4', name: 'Meera Iyer', role: 'QA Engineer', bio: 'Finds the edge case you were sure did not exist.' },
  { id: 'u5', name: 'Rohan Gupta', role: 'Engineering Manager', bio: 'Unblocks the team and guards the roadmap.' },
  { id: 'u6', name: 'Sara Khan', role: 'Data Analyst', bio: 'Measures whether the new loader really feels faster.' },
];

export const SCENARIOS = {
  fast: { label: 'Fast (100 ms)', delay: 100 },
  slow: { label: 'Slow (2 s)', delay: 2000 },
  empty: { label: 'Empty (1.2 s)', delay: 1200 },
  error: { label: 'Error (1.5 s)', delay: 1500 },
};

export function fetchUsers({ scenario = 'slow', signal } = {}) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      if (scenario === 'error') reject(new Error('Could not load users. Please try again.'));
      else resolve(scenario === 'empty' ? [] : USERS);
    }, SCENARIOS[scenario]?.delay ?? 1000);

    signal?.addEventListener(
      'abort',
      () => {
        clearTimeout(timer);
        reject(new DOMException('Request aborted', 'AbortError'));
```

```
      },
      { once: true },
    );
  });
}
```

`src/hooks/useUsers.js`

```
import { useCallback, useEffect, useState } from 'react';
import { fetchUsers } from '../api/usersApi';

const LOADING = { status: 'loading', data: [], error: null };

export function useUsers(scenario) {
  const [state, setState] = useState(LOADING);
  const [reloadKey, setReloadKey] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setState(LOADING);

    fetchUsers({ scenario, signal: controller.signal })
      .then((data) => setState({ status: 'success', data, error: null }))
      .catch((err) => {
        // An aborted request belongs to an old scenario: ignore it.
        if (err.name !== 'AbortError') setState({ status: 'error', data: [], error: err.message });
      });

    return () => controller.abort();
  }, [scenario, reloadKey]);

  const reload = useCallback(() => setReloadKey((k) => k + 1), []);
  return { ...state, reload };
}
```

`src/hooks/useDeferredLoading.js`

```
import { useEffect, useRef, useState } from 'react';

// Show the skeleton only if loading lasts longer than `delay`,
// and once shown keep it for at least `minDuration` so it never flickers.
export function useDeferredLoading(loading, { delay = 200, minDuration = 500 } = {}) {
  const [show, setShow] = useState(false);
  const shownAt = useRef(0);
```

```
  useEffect(() => {
    if (loading) {
      if (show) return;
      const timer = setTimeout(() => {
        shownAt.current = Date.now();
        setShow(true);
      }, delay);
      return () => clearTimeout(timer);
    }

    if (!show) return;
    const remaining = minDuration - (Date.now() - shownAt.current);
    const timer = setTimeout(() => setShow(false), Math.max(0, remaining));
    return () => clearTimeout(timer);
  }, [loading, show, delay, minDuration]);

  return show;
}
```

`src/components/Skeleton.jsx`

```
import './Skeleton.css';

// Generic placeholder block. Decorative only: the parent announces "loading".
export function Skeleton({ variant = 'text', width, height, lines = 1, animation = 'shimmer',
className = '' }) {
  const cls = `skeleton skeleton--${variant} skeleton--${animation} ${className}`.trim();

  if (variant === 'text' && lines > 1) {
    return (
      <span className="skeleton-lines" aria-hidden="true">
        {Array.from({ length: lines }, (_, i) => (
          // Last line shorter so it reads like a real paragraph.
          <span key={i} className={cls} style={{ width: i === lines - 1 ? '60%' : width }} />
        ))}
      </span>
    );
  }

  return <span className={cls} style={{ width, height }} aria-hidden="true" />;
}
```

src/components/Skeleton.css

```
.skeleton {
  display: block;
  position: relative;
  overflow: hidden;
  background: var(--sk-base);
  border-radius: 4px;
}

/* Text bar takes exactly one line-height of the inherited font, then is scaled down visually.
   transform does not affect layout, so the box equals the real text line: no shift on swap. */
.skeleton--text {
  height: auto;
  transform: scale(1, 0.6);
  transform-origin: 0 55%;
}

.skeleton--text::before {
  content: '\00a0';
}

.skeleton--circle {
  border-radius: 50%;
  flex-shrink: 0;
}

.skeleton--rect {
  border-radius: 8px;
}

.skeleton-lines {
  display: block;
}

/* Shimmer moves a pseudo-element with transform: compositor-only, no repaint per frame. */
.skeleton--shimmer::after {
  content: '';
  position: absolute;
  inset: 0;
  transform: translateX(-100%);
  background: linear-gradient(90deg, transparent, var(--sk-highlight), transparent);
  animation: sk-shimmer 1.4s ease-in-out infinite;
}
```

```
.skeleton--pulse {
  animation: sk-pulse 1.5s ease-in-out infinite;
}

@keyframes sk-shimmer {
  to {
    transform: translateX(100%);
  }
}

@keyframes sk-pulse {
  50% {
    opacity: 0.5;
  }
}

@media (prefers-reduced-motion: reduce) {
  .skeleton--shimmer::after,
  .skeleton--pulse {
    animation: none;
  }
}
```

`src/components/UserCard.jsx`

```
const initials = (name) =>
  name
    .split(' ')
    .map((part) => part[0])
    .join('')
    .slice(0, 2);

export function UserCard({ user }) {
  return (
    <article className="card">
      <div className="card__header">
        <div className="avatar" aria-hidden="true">{initials(user.name)}</div>
        <div className="card__titles">
          <h2 className="card__name">{user.name}</h2>
          <p className="card__role">{user.role}</p>
        </div>
      </div>
      <p className="card__bio">{user.bio}</p>
    </article>
```

```
  );
}
```

`src/components/UserCardSkeleton.jsx`

```
import { Skeleton } from './Skeleton';

// Mirrors UserCard box for box (same .card classes) so the swap causes no layout shift.
export function UserCardSkeleton({ animation }) {
  return (
    <article className="card" aria-hidden="true">
      <div className="card__header">
        <Skeleton variant="circle" width={48} height={48} animation={animation} />
        <div className="card__titles">
          <Skeleton className="card__name" width="55%" animation={animation} />
          <Skeleton className="card__role" width="35%" animation={animation} />
        </div>
      </div>
      <div className="card__bio">
        <Skeleton lines={2} width="100%" animation={animation} />
      </div>
    </article>
  );
}
```

`src/App.jsx`

```
import { useState } from 'react';
import { SCENARIOS } from './api/usersApi';
import { useUsers } from './hooks/useUsers';
import { useDeferredLoading } from './hooks/useDeferredLoading';
import { UserCard } from './components/UserCard';
import { UserCardSkeleton } from './components/UserCardSkeleton';
import './App.css';

const PAGE_SIZE = 6; // skeleton count = expected result count, so the grid does not jump

export default function App() {
  const [scenario, setScenario] = useState('slow');
  const [animation, setAnimation] = useState('shimmer');
  const { status, data, error, reload } = useUsers(scenario);
  const showSkeleton = useDeferredLoading(status === 'loading');
  const busy = status === 'loading' || showSkeleton;
```

```
let content = null; // loading but under the delay: render nothing, avoids a flash
if (showSkeleton) {
  content = (
    <ul className="grid">
      {Array.from({ length: PAGE_SIZE }, (_, i) => (
        <li key={i}>
          <UserCardSkeleton animation={animation} />
        </li>
      ))}
    </ul>
  );
} else if (status === 'error') {
  content = (
    <div className="message message--error" role="alert">
      <p>{error}</p>
      <button type="button" onClick={reload}>
        Retry
      </button>
    </div>
  );
} else if (status === 'success' && data.length === 0) {
  content = <p className="message">No users found.</p>;
} else if (status === 'success') {
  content = (
    <ul className="grid">
      {data.map((user) => (
        <li key={user.id}>
          <UserCard user={user} />
        </li>
      ))}
    </ul>
  );
}

return (
  <main className="app">
    <h1>Team members</h1>

    <div className="controls">
      <label>
        Scenario
        <select value={scenario} onChange={(e) => setScenario(e.target.value)}>
          {Object.entries(SCENARIOS).map(([key, { label }]) => (
            <option key={key} value={key}>
              {label}
            </option>
```

```
            ))}
          </select>
        </label>
        <label>
          Animation
          <select value={animation} onChange={(e) => setAnimation(e.target.value)}>
            <option value="shimmer">Shimmer</option>
            <option value="pulse">Pulse</option>
            <option value="none">None</option>
          </select>
        </label>
        <button type="button" onClick={reload} disabled={busy}>
          Reload
        </button>
      </div>

      <section className="results" aria-busy={busy}>
        <span className="sr-only" role="status">
          {busy ? 'Loading users…' : ''}
        </span>
        {content}
      </section>
    </main>
  );
}
```

`src/App.css`

```
:root {
  --bg: #f6f7f9;
  --surface: #ffffff;
  --text: #1c1f24;
  --muted: #5f6672;
  --border: #e3e6eb;
  --accent: #4f46e5;
  --danger: #b42318;
  --sk-base: #e6e8ec;
  --sk-highlight: rgba(255, 255, 255, 0.7);
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  color: var(--text);
  background: var(--bg);
}

@media (prefers-color-scheme: dark) {
  :root {
```

```
    --bg: #14161a;
    --surface: #1e2127;
    --text: #e8eaed;
    --muted: #9aa1ac;
    --border: #2d3139;
    --accent: #8b85ff;
    --danger: #ff8a80;
    --sk-base: #2a2e35;
    --sk-highlight: rgba(255, 255, 255, 0.08);
  }
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--bg);
}

.app {
  max-width: 960px;
  margin: 0 auto;
  padding: 24px 16px;
}

.controls {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  align-items: end;
  margin-bottom: 20px;
}

.controls label {
  display: grid;
  gap: 4px;
  font-size: 14px;
  color: var(--muted);
}

select,
button {
  font: inherit;
  padding: 6px 12px;
```

```
  border-radius: 6px;
  border: 1px solid var(--border);
  background: var(--surface);
  color: var(--text);
}

button {
  cursor: pointer;
  background: var(--accent);
  border-color: var(--accent);
  color: #fff;
}

button:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.results {
  min-height: 420px; /* reserve space so the blank pre-delay phase does not collapse the page */
}

.grid {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 16px;
}

/* Shared by UserCard and UserCardSkeleton: same box, same size. */
.card {
  height: 100%;
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: 12px;
  background: var(--surface);
}

.card__header {
  display: flex;
  gap: 12px;
  align-items: center;
  margin-bottom: 12px;
}
```

```
.card__titles {
  flex: 1;
  min-width: 0;
}

.avatar {
  width: 48px;
  height: 48px;
  border-radius: 50%;
  flex-shrink: 0;
  display: grid;
  place-items: center;
  font-weight: 600;
  background: var(--accent);
  color: #fff;
}

.card__name {
  margin: 0;
  font-size: 16px;
  line-height: 1.4;
}

.card__role {
  margin: 0;
  font-size: 14px;
  line-height: 1.4;
  color: var(--muted);
}

.card__bio {
  margin: 0;
  font-size: 14px;
  line-height: 1.5;
  color: var(--muted);
}

.message {
  padding: 24px;
  text-align: center;
  color: var(--muted);
}

.message--error {
  color: var(--danger);
```

```
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

---

12. Wrap-up

Edge cases handled

- Fast responses skip the skeleton; medium ones don't flicker.
- No layout shift: shared classes + line-height-based text bars + fixed avatar size + page-size count.
- Error with Retry, empty state, aborted stale requests, unmount safety.
- Reduced motion, dark mode, screen reader announcement, decorative blocks hidden.

Trade-offs to mention

- **Hand-written composite skeleton vs auto-generated** from the real component: hand-written is explicit and fast, but must be kept in sync when the card changes.
- **Fixed skeleton count (6)** vs real count: we don't know the count before the response. Page size is a good guess; for "load more" you'd use the previous count.

- **200 ms blank phase** on first load: avoids flash, but the user sees nothing for 200 ms. For first paint you could use `delay: 0`.
- **Shimmer vs pulse**: shimmer shows "progress" better; pulse is simpler and even cheaper.
- **Clear data on reload** vs keep stale data with an overlay: clearing is simpler; stale-while-revalidate is better UX for refreshes.

With more time

- Tests: React Testing Library + fake timers for `useDeferredLoading` (100 ms, 250 ms, 2 s cases), and an axe check.
- `<Suspense fallback={<UserCardSkeleton/>}>` with a data library (React Query / `use()`), streaming SSR with skeleton fallbacks.
- One shared shimmer across all blocks (`background-attachment: fixed` or a single overlay) so every block shimmers in sync.
- More variants (`rounded`, `table row`, `image` with aspect-ratio) and a `SkeletonGroup` that sets `animation` once via context.
- Measure CLS with the web-vitals library in production.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why a skeleton instead of a spinner?

A skeleton previews the *shape* of the content, so the page feels faster and the eye already knows where things will appear. Spinners fit short, unknown-shape actions (button submit). For content areas with a known layout, skeletons usually feel faster.

Q2. When should you NOT show a skeleton?

When the response is very fast (< 200 ms: it would just flash), for tiny inline updates, or when you can show cached/stale data instead. Also not for actions where the user expects a progress indicator (file upload → progress bar).

Q3. How do you know how many skeleton items to show?

Use the expected count: page size, previous result count, or a count from the server (e.g. total in a header). If unknown, show enough to fill the viewport. Too many causes a bigger collapse when fewer items arrive.

State design

Q4. Why one `status` enum instead of `isLoading` and `isError` booleans?

Booleans allow impossible states (`isLoading && isError`). One enum makes each render branch exclusive and easier to test: `loading | success | error`.

Q5. Why is `shownAt` a ref and not state?

It's read only inside the effect to compute the remaining time; it never changes what is rendered. State would cause an extra render for nothing.

Q6. Why not store `isEmpty` or `showSkeleton` in `useUsers`?

`isEmpty` is derived from `status` and `data`. `showSkeleton` is a *presentation* concern (timing for the eye), not a data concern. Keeping it in its own hook lets any loader reuse it.

Challenge 1: layout shift

Q7. How do you verify there's no layout shift?

Compare `offsetHeight` of skeleton vs real elements (here 22 / 20 / 42 px both ways), watch the Performance panel's "Layout shift" track, or use `PerformanceObserver` with `layout-shift` / the web-vitals CLS metric.

Q8. Why `transform: scale` and not just a smaller `height`?

`height` changes layout, so the box no longer equals the text line. `transform` is visual only, so the box stays one line-height tall while the bar *looks* thinner, like real glyphs.

Q9. What if the real content has variable height (long bios)?

Match the common case (2 lines) and accept a small change for outliers, or clamp the real text (`line-clamp: 2`) so both are always identical. For images, reserve space with `aspect-ratio` or width/height attributes.

Challenge 2: timing

Q10. Walk me through a 250 ms response.

At 200 ms the skeleton appears (`shownAt = 200`). At 250 ms loading ends; remaining = 500 − 50 = 450 ms, so it hides at 700 ms. Without the minimum it would flash for 50 ms.

Q11. Doesn't the minimum duration make the app slower?

Slightly, by up to 500 ms in a rare window, but perceived speed improves because there is no flicker. React itself throttles how often Suspense fallbacks are revealed for the same reason. The numbers are tunable options.

Q12. How would you test `useDeferredLoading`?

`vi.useFakeTimers()`, render a test component with `loading=true`, advance 100 ms → expect false; advance to 250 ms → true; set `loading=false`, advance 400 ms → still true; advance 100 ms more → false.

Challenge 3: animation performance

Q13. Why animate `transform` instead of `background-position`?

`background-position` triggers a repaint of each block every frame. `transform` on a separate layer runs on the compositor thread: no layout and no paint, smooth even if the main thread is busy with JS.

Q14. What if there are 500 skeletons on screen?

500 animated layers costs GPU memory. Options: render only what's visible (virtualize the list), use `pulse` (opacity, very cheap), or a single shimmer overlay over the whole list instead of one per block.

Q15. How do you make all blocks shimmer in sync?

Give every block the same gradient with `background-attachment: fixed` so the gradient is relative to the viewport, or use one overlay element animated across the container.

Challenge 4: accessibility and races

Q16. Why `aria-hidden` on the skeleton blocks?

They carry no information. Without it a screen reader may walk through dozens of empty elements. One `role="status"` message plus `aria-busy` on the region is enough.

Q17. Why does `aria-busy` matter?

It tells assistive tech that the region is being updated, so it can wait until the update finishes before announcing changes, instead of reading half-rendered content.

Q18. What happens if the user switches scenario three times quickly?

Each change re-runs the effect; the cleanup aborts the previous controller, so only the last request can call `setState`. Without the abort, an older slow response could land last and overwrite newer data.

Implementation steps

Q19. How would you do this with Suspense?

Wrap the list in `<Suspense fallback={<UserListSkeleton/>}>` and read data with a Suspense-enabled source (`use(promise)`, React Query `useSuspenseQuery`). The skeleton becomes the fallback; error handling moves to an Error Boundary. `useTransition` keeps old content during refetches.

Q20. How would a design system expose this?

A `Skeleton` primitive (variants, size, animation), a `SkeletonGroup`/provider for shared animation and colors via context, and composite skeletons living next to each component (`Card.Skeleton`), so they change together.

"What if" and "why not X?"

Q21. What if the API is slow AND the user is on a slow device?

The skeleton is pure CSS, so it costs no JS per frame even on a busy main thread. Going further, server-render the skeleton markup into the HTML so the user sees the page structure before the JS bundle even loads.

Q22. Why not use a library like `react-loading-skeleton`?

For an interview, it shows nothing. In production it's fine, but it still needs the same thinking: layout match, timing, accessibility. Our version is ~30 lines of JS and CSS with no dependency.

Q23. What if we refresh data that is already on screen?

Don't swap back to skeletons: that throws content away. Keep the stale data, show a subtle progress bar or reduced opacity, and swap when new data arrives (stale-while-revalidate).

Q24. What if the request takes 30 seconds?

A skeleton that shimmers forever looks broken. Add a timeout (abort after N seconds → error with Retry), or after ~10 s show a "Still loading…" message next to the skeleton.